Model Selection and Architecture for AI Video Script Generator

EPIC 1

---

1. Introduction

The objective of this project is to develop an AI-powered Video Script Generator that can automatically create structured video scripts based on user inputs such as topic, tone, audience, platform, and duration.

To achieve this, selecting the correct Generative AI model and designing an efficient application architecture is essential. This report explains the model selection process, system architecture, and development environment setup.

---

2. Research and Selection of Appropriate GenAI Model

2.1 Requirements of the Model

The AI model must:

- Generate coherent and structured video scripts
- Understand prompts like topic, tone, audience, and duration
- Provide fast response time for real-time usage
- Be cost-efficient for student/development use
- Support API integration with Python backend

2.2 Evaluated Options

Model	Strengths	Limitations
OpenAI GPT models	High-quality output, very reliable	Paid API, usage cost
Google Gemini	Strong reasoning and multimodal ability	Slightly complex integration
Anthropic Claude	Good long-form content generation	Limited free access
LLaMA-based models	Open-source, customizable	Requires high compute resources
Groq LLM API (LLaMA/Mixtral hosted)	Extremely fast inference, easy API, cost-efficient	Depends on Groq availability

---

2.3 Final Selected Model

Selected Model: Groq API (LLaMA / Mixtral-based models)

Reasons for Selection:

- Very high-speed inference (ideal for real-time script generation)
- Simple REST API integration with Python (Flask backend)

- Supports structured prompt-based outputs
- Free/low-cost access suitable for student projects
- Reliable performance for text generation tasks

Use in Project:

The model is used to:

- Generate video scripts from structured prompts
- Adjust tone (formal, humorous, motivational, etc.)
- Adapt content for platforms (YouTube, Instagram, reels, etc.)

---

3. Architecture of the Application

3.1 Overview of System Architecture

The system follows a **client-server architecture** with AI integration.

```
User Interface (Frontend)
        ↓
Flask Backend (Python Server)
        ↓
Prompt Processing Module
        ↓
Groq LLM API (AI Model)
        ↓
Generated Video Script
        ↓
Response sent back to User
```

3.2 Components Description

1. User Interface (Frontend)

- Accepts input from user:
  - Topic
  - Audience type
  - Tone (formal, motivational, humorous)
  - Platform (YouTube, Instagram, etc.)
  - Duration
- Can be built using simple HTML/CSS or basic web form

---

2. Flask Backend (Core Logic Layer)

- Built using Python Flask framework
- Handles:
  - API requests from frontend
  - Input validation
  - Prompt construction
  - Communication with Groq API

---

3. Prompt Engineering Module

This is a key part of the system.

Example prompt structure:

```
Write a video script on {topic} for {audience}.
Tone: {tone}
Platform: {platform}
Duration: {duration}
Include hook, main content, and conclusion.
```

---

4. AI Model Layer (Groq API)

- Receives structured prompt
- Processes it using LLaMA/Mixtral model
- Returns generated script

---

5. Output Module

- Displays generated script on UI
- Optionally allows download or copy feature

3.3 Architecture Type

- Client-Server Architecture
- API-Based AI Integration
- Modular design (Frontend, Backend, AI Layer separated)

---

4. Development Environment Setup

4.1 Tools Required

Tool	Purpose
Python (3.10+)	Backend development
VS Code	Code editor
Flask	Web framework
pip	Package manager
Groq API Key	AI model access
Git (optional)	Version control

4.2 Installation Steps

Step 1: Install Python

- Install Python 3.10 or above
- Enable:
    - “Add Python to PATH”

Step 2: Install Flask

```
pip install flask
```

Step 3: Install Groq SDK

```
pip install groq
```

Step 4: Set Up Project Folder

Example structure:

```
AI-Video-Script-Generator/
│
├── app.py
├── templates/
│     └── index.html
├── static/
│     └── style.css
└── requirements.txt
```

Step 5: Configure API Key

Store Groq API key securely:

```
import os
os.environ["GROQ_API_KEY"] = "your_api_key_here"
```

Step 6: Run Flask Server

```
python app.py
```

---

5. Conclusion

The combination of **Groq LLM API, Flask backend, and structured prompt engineering** provides an efficient and scalable solution for building an AI Video Script Generator.

The selected architecture ensures:

- Fast response time
- Simple integration
- Modular development
- Easy scalability for future improvements (like voice-over, video generation, etc.)